

ECDSA

Hertrich, Samuel
(00127810)

Lotz, Moritz
(00127394)

Taffertshofer, Vincent
(00127888)

Contents

1	Introduction	1
2	Brief history of ECDSA	1
3	Use Cases	2
3.1	Healthcare and Public Services	2
3.1.1	Electronic Health (E-health)	2
3.1.2	Electronic Governance (E-governance)	3
3.2	Financial Services	3
3.2.1	Electronic Banking (E-banking)	3
3.2.2	Electronic Commerce (E-commerce)	4
3.3	Transportation and Infrastructure	4
3.3.1	Electronic-Vehicular Networks (VANETs)	4
3.4	Security Protocols and Standards	5
3.4.1	Secure Shell (SSH)	5
3.4.2	Transport Layer Security (TLS)	5
3.5	Digital Documentation and Cryptography	5
3.5.1	Signing PDF Documents	5
3.5.2	Blockchain	6
4	Comparison of ECDSA with DSA and RSA	6
4.1	RSA (Rivest–Shamir–Adleman)	6
4.2	DSA (Digital Signature Algorithm)	7
4.3	ECDSA (Elliptic Curve Digital Signature Algorithm)	7
4.4	Comparative Summary	7
5	Cryptographic Foundations of ECDSA	8
5.1	The Discrete Logarithm Problem	8
5.2	Elliptic curve group	8
5.2.1	Example curve <i>secp256k1</i>	9
5.2.2	Elliptic curve group operation	9
5.2.3	Calculating R	10
5.3	Key Pair Generation	11
5.4	Signature Generation	12
5.5	Signature Verification	13
5.6	ECDLP vs DLP - why it is more secure	14
5.6.1	Baby Step Giant Step	14
5.6.2	Pollard’s Rho-Algorithm	15
6	Security Risks	16
6.1	Poor Random Number Generator	17
6.1.1	Repetition	17
6.1.2	Predictability	18

6.1.3	Size	18
6.1.4	Bias	18
6.1.5	Examples	19
6.1.6	Solution	20
6.2	Timing Attack	20
6.2.1	Key Extraction	20
6.2.2	Solution	21
7	Conclusion	21

Abstract

Elliptic Curve Digital Signature Algorithm (ECDSA) stands as a cornerstone in modern cryptographic systems, offering secure and efficient digital signatures with relatively short key sizes. This report provides broad overview of ECDSA, covering its history, applications, underlying principles, procedural outline, potential attacks against it as well as some security concerns. Through this analysis, the report aims to provide the readers with a basic understanding of ECDSA and its significance in our long digitalized world of already high but steadily increasing security requirements.

1 Introduction

In today's digital landscape, the security of electronic transactions and communications is paramount. Digital signatures are critical in verifying the authenticity and integrity of data, ensuring secure and trustworthy interactions in virtual environments. Among the various digital signature algorithms, the Elliptic Curve Digital Signature Algorithm (ECDSA) distinguishes itself through its combination of efficiency and robust security features. This report provides an in-depth examination of ECDSA, outlining its foundational principles, operational mechanisms, and diverse applications.

ECDSA's significance is particularly evident in high-security environments requiring both strong security and operational efficiency. Its ability to provide security comparable to RSA and DSA while using significantly smaller key sizes reduces computational demands and enhances transaction speeds, making it a preferred choice for modern cryptographic applications.

The mathematical foundation of ECDSA is rooted in elliptic curve cryptography, which leverages the complexity of the Elliptic Curve Discrete Logarithm Problem (ECDLP) to provide robust security. This report also explores potential security challenges, including vulnerabilities, to ensure a comprehensive understanding of ECDSA's reliability.

By offering a comprehensive overview of ECDSA in the context of digital signature technology, this report aims to highlight its crucial role in securing digital identities and transactions. It seeks to equip readers with a thorough understanding of ECDSA's practical applications and theoretical underpinnings, emphasizing its importance in the ever-evolving field of digital security.¹

2 Brief history of ECDSA

All information in this section is derived from [10]. Initially, ChatGPT generated the content, which was then revised to ensure alignment with the source, removing any details not found in the original material. Subsequently, some parts were manually simplified for clarity.

¹This introduction was initially generated using ChatGPT and then revised to ensure it accurately reflects the content of the report.

The Elliptic Curve Digital Signature Algorithm (ECDSA), as an elliptic curve analogue of the Digital Signature Algorithm (DSA), has a rich history marked by significant milestones in cryptographic development. In 1985 the mathematicians Neal Koblitz and Victor Miller introduced the foundational concept of Elliptic Curve Cryptosystems (ECC). The security of ECC's is based on the Elliptic Curve Discrete Logarithm Problem (ECDLP) which is much more difficult than the traditional discrete logarithm problem. Therefore these systems provide enhanced security with smaller key sizes.

In response to NIST's (National Institute of Standards and Technology) request for public comments on their initial draft for the Digital Signature Standard (DSS) ECDSA was first proposed by Scott Vanstone in 1992. This proposal set the stage for ECDSA's journey through various standardization processes.

ECDSA was first recognized as an International Standards Organization (ISO) standard (ISO 14888-3) in 1998. In the following year, it was accepted as a standard by the American National Standards Institute (ANSI) under ANSI X9.62. The recognition of the ECDSA continued to grow when it was adopted as a standard by the Institute of Electrical and Electronics Engineers (IEEE) under IEEE 1363-2000 and as a Federal Information Processing Standard (FIPS 186-2).

Unlike algorithms that rely on the ordinary discrete logarithm or integer factorization problems, the elliptic curve discrete logarithm problem has no known subexponential-time algorithms, resulting in a much higher strength-per-key-bit ratio. This makes ECDSA a highly secure choice for cryptographic applications.

The widespread use and standardization of ECDSA underline its importance and effectiveness in digital signature processes, especially in environments where computing power, storage space and bandwidth are limited. ECDSA's ability to provide strong security with efficient implementation has made it a preferred choice in various cryptographic standards and applications.

3 Use Cases

Sections 3.1 through 3.3 are based on information from source [1], while section 3.4 is derived from source [19]. It should also be noted that the text in this section was generated by ChatGPT and then compared to sources that were selected beforehand. Footnotes detail the modifications made to the generated text.

3.1 Healthcare and Public Services

3.1.1 Electronic Health (E-health)

Electronic health (e-health) facilitates the sharing of medical records across various healthcare settings, such as hospitals, clinics, and homes. These services, which include electronic health records (EHR), electronic medical records (EMR), and personal health records (PHR), enable healthcare providers and patients to access patient data electronically - anywhere and anytime - improving the quality of care, disease diagnosis, and remote medical surveillance.

Nevertheless, the major obstacle impeding the widespread adoption of these systems is the protection of patient data ². Medical information transmitted from devices like WSNs (Wireless Sensor Networks ³), laptops, and phones to e-health servers is particularly vulnerable to cyberattacks and intrusions, as the Internet and wireless networks are not fully secure environments. Data privacy is crucial in e-health, requiring robust security measures such as authorization policies, encryption algorithms, and secure digital signatures to protect medical repositories from malicious threats. This is vital because healthcare data is among the most sensitive and frequently targeted by cybercriminals, as evidenced by numerous security breaches impacting healthcare institutions globally. Many modern projects in the e-health sector implement the ECDSA algorithm for its high security and efficient performance to further bolster the protection of sensitive data ⁴.

3.1.2 Electronic Governance (E-governance)

Electronic governance (e-governance) leverages technologies like the Internet, SMS, and email to streamline government services for citizens, businesses, and government bodies. It encompasses a variety of transactions including government to citizens (G2C), business (G2B), employees (G2E), and intergovernmental (G2G) interactions, aiming to enhance transparency, flexibility, and manageability of services.

However, the adoption of e-governance also presents significant security challenges, with platforms exposed to various threats such as hacking and natural disasters. To mitigate these risks, comprehensive security measures including server, network, and data security are implemented. Advanced cryptographic methods like Elliptic Curve Cryptography (ECC) and the Elliptic Curve Digital Signature Algorithm (ECDSA) are particularly crucial for protecting sensitive data and ensuring secure communications within and between government entities. These technologies are key in maintaining the integrity and privacy of e-governance transactions, securing them against a wide array of cyber threats.

3.2 Financial Services

3.2.1 Electronic Banking (E-banking)

Electronic banking (e-banking) utilizes various platforms such as online banking, mobile banking, credit cards, and ATMs, all of which require robust security mechanisms to protect customer accounts from sophisticated cyber threats. Traditional authentication methods like usernames and passwords are increasingly proving to be inadequate against these modern security challenges. To enhance security, there has been a shift towards

²ChatGPT's repetitive sentence structure was modified for a more natural flow in the context of the text.

³Neither the source nor ChatGPT provided the meaning of the acronym, so it was added manually afterwards.

⁴This sentence was generated later, as it was not initially included by ChatGPT, but was added afterwards due to its importance for this seminar work.

adopting advanced cryptographic technologies, such as the Elliptic Curve Digital Signature Algorithm (ECDSA), which strengthens the authentication processes.

By implementing these technologies, e-banking systems can better prevent risks associated with phishing, keylogging, and other cyber threats, ensuring secure and reliable transactions for users across various digital banking platforms. ⁵

3.2.2 Electronic Commerce (E-commerce)

Electronic-commerce (e-commerce) heavily relies on the Internet to facilitate transactions between governments, companies, and consumers. While the Internet has significantly boosted the convenience and reach of e-commerce, it has also exposed it to substantial security risks such as hacking, credential replay attacks, and DDoS attacks. These vulnerabilities underscore the need for robust security mechanisms to protect sensitive information and ensure secure transactions.

To address these security challenges, e-commerce platforms have adopted currency signing systems that leverage advanced cryptographic technologies, such as the Elliptic Curve Digital Signature Algorithm (ECDSA) ⁶. This approach significantly enhances key security features, including authentication, integrity, and non-repudiation, ensuring robust protection for online transactions. ⁷

3.3 Transportation and Infrastructure

3.3.1 Electronic-Vehicular Networks (VANETs)

Vehicular ad hoc networks (VANET) significantly improve road safety and efficiency by enabling dynamic communication between vehicles and road infrastructure. These networks leverage self-regulation and distributed communication environments to effectively manage traffic, prevent accidents, and enforce compliance with road laws. VANET systems operate primarily in two modes: vehicle-to-vehicle (V2V) and vehicle-to-infrastructure (V2I), functioning as a specialized form of mobile ad hoc networks (MANET) with structured interactions.

Security poses a major challenge in VANETs due to the openness of the communication medium, which exposes the network to various cyber threats. To safeguard data, VANETs employ encryption and digital signature protocols, notably the Elliptic Curve Digital Signature Algorithm (ECDSA). This cryptographic method helps authenticate messages, ensuring their integrity and non-repudiation while being efficient enough for constrained resource environments. The implementation of ECDSA and its effectiveness in VANETs depends crucially on the selection of cryptographic curves and the balance between key size and the efficiency of signature processes.

⁵Here, ChatGPT provided too much information about examples, so the unnecessary part was removed.

⁶ChatGPT wrote that the e-commerce apps implemented the algorithms, but according to the source, they only implemented currency signing systems.

⁷In this instance, ChatGPT wrote more than needed, so the text was shortened.

3.4 Security Protocols and Standards

3.4.1 Secure Shell (SSH)

Secure Shell (SSH) employs Elliptic Curve Cryptography (ECC) in three crucial ways to enhance network security and integrity, for example in the SSH-2 protocol. Firstly, SSH-2 utilizes the Diffie-Hellman key exchange algorithm to generate equivalent keys⁸. During this process, each server is equipped with a unique host key, which is fundamental for the server-client authentication phase. This key is sent to the client during the key exchange, allowing the client to verify its authenticity by comparing the key's fingerprint against a stored value.

Moreover, the server substantiates its identity by signing a transcript of the key exchanges. This signature, potentially created using the Elliptic Curve Digital Signature Algorithm (ECDSA), certifies the integrity of the recorded exchanges and ensures that the communication remains secure. Additionally, this use of ECDSA is not limited to server authentication. Clients also employ ECDSA public keys to authenticate each other.⁸

3.4.2 Transport Layer Security (TLS)

Elliptic curves play a significant role in the application of the Transport Layer Security (TLS) protocol, as detailed in RFC 4492. This protocol specification includes the use of the Diffie-Hellman key exchange algorithm, which is based on elliptic curves. Known as ECDH (Elliptic Curve Diffie-Hellman), this method of key exchange utilizes keys of two distinct types. The first type is long-term keys, which allow for multiple key exchanges to reuse the same keys. The second type is short-lived keys, where each key exchange leads to the generation of new keys.⁸

Additionally, TLS certificates contain a public key used by servers to authenticate themselves to clients. This authentication is critical for establishing a secure and trusted connection, ensuring that data transmitted over the network remains confidential and integral.

3.5 Digital Documentation and Cryptography

3.5.1 Signing PDF Documents

The Portable Document Format (PDF) is widely used for document management and increasingly for digital signatures, as more aspects of life transition to digital formats. Despite their growing significance, it wasn't until PDF 2.0 that the PDF standards began to support the Elliptic Curve Digital Signature Algorithm (ECDSA), a method ensuring more secure digital signatures.

Adobe Acrobat, a leading PDF application, has been a pioneer in this area, supporting ECDSA signatures since version 11.0. Adobe's early adoption of ECDSA has served as

⁸The content provided by ChatGPT exceeded the source material and was therefore condensed.

a de facto reference for other PDF tools and applications, facilitating the use of modern cryptographic techniques for signing PDF documents securely. [7]

3.5.2 Blockchain

Blockchain technology, famously represented by Bitcoin, facilitates direct peer-to-peer transactions without the need for intermediaries. This decentralized digital currency system records transactions on a digital ledger, where each entry is linked in a chain secured by the Elliptic Curve Digital Signature Algorithm (ECDSA). In this system, private keys function as accounts on the blockchain, crucial for verifying each transaction. [19]

4 Comparison of ECDSA with DSA and RSA

This section primarily relies on [20] as a source, with the exception of section 4.3, which incorporates one additional source. It is also important to note that the text in this section was generated using ChatGPT and subsequently verified with pre-selected sources for accuracy. The footnotes detail the utilization of the generated text in this section.

The three most popular digital signature algorithms RSA, DSA and ECDSA can be used to send information through insecure channels like open networks to ensure data integrity and authenticity. Each of these algorithms has unique characteristics which are suitable for different scenarios, depending on requirements like computational efficiency, key size, and security level. ⁹

4.1 RSA (Rivest–Shamir–Adleman)

RSA is an algorithm for public-key cryptography invented by Ron Rivest, Adi Shamir, and Leonard Adleman at MIT in 1977. It is the first known algorithm that can be used for both signing and encryption. RSA uses the approach of factoring large integers and employs a pair of keys: a public key for encryption (or verifying a signature) and a private key for decryption (or generating a signature). The signature generation process in RSA involves creating a hash of the message and producing a signature with the sender's private key. Verification is achieved by decrypting the signature with the sender's private key and comparing it with a newly generated hash of the message that uses the same hash algorithm. ¹⁰

RSA's security strength largely depends on the key size. Computational requirements for RSA are relatively high, especially for key generation and encryption processes, making it less efficient for devices with limited processing power.

⁹This paragraph was first structured using ChatGPT, then refined to ensure complete alignment with the provided source document, omitting any content not explicitly found in the source.

¹⁰The paragraph provided by ChatGPT was rewritten and simplified.

4.2 DSA (Digital Signature Algorithm)

Developed by the National Institute of Standards and Technology (NIST) in 1991, DSA is specifically designed for digital signatures and does not support encryption. DSA (Digital Signature Algorithm) involves selecting a set of mathematical parameters to create a unique pair of keys: a private key for signing messages and a public key for others to verify those signatures. To sign a message, a random number is generated and used with the private key to produce a signature composed of two numbers. When verifying a signature, calculations involving the public key and the message's hash are performed to ensure the signature was made with the corresponding private key and that it matches the content of the message.¹¹

DSA's key advantage is its efficiency in generating signatures, making it suitable for environments where signing speed is critical. However, its verification process is slower compared to RSA, which can be a drawback in scenarios requiring rapid signature verification.

4.3 ECDSA (Elliptic Curve Digital Signature Algorithm)

ECDSA is a variant of the DSA that uses elliptic curve cryptography, offering equivalent security to RSA and DSA but with significantly smaller key sizes [20]. This results in faster computations and reduced resource consumption [12], making ECDSA highly suitable for resource-constrained environments.

The ECDSA signature generation process starts by establishing elliptic curve parameters and producing a key pair consisting of a private key and a public key. A hash of the message is computed, and the signature is generated using elliptic curve point multiplication and modular arithmetic using a random selected integer¹². In the verification process, the signature and public key are received, the hash of the message is recomputed, and the signature is validated by performing elliptic curve computations and checking if the resulting value matches the signature components. [20]

4.4 Comparative Summary

In summary, RSA, DSA, and ECDSA each present distinct advantages and trade-offs, tailored to varying requirements in digital security practices. RSA is versatile, supporting both encryption and signature functionalities, making it ideal for comprehensive security applications, though it demands significant computational resources. DSA, specialized for digital signatures without encryption capabilities, excels in environments where rapid signature generation is crucial, despite a slower verification process. ECDSA leverages elliptic curve cryptography to provide the same level of security as RSA and DSA but with much smaller key sizes, enhancing performance and making it exceptionally suitable for

¹¹The explanation of the process provided by ChatGPT was initially too brief, prompting a request for a more detailed elaboration.

¹²ChatGPT's overly technical sentence was simplified manually after a confusing revision.

resource-limited settings. Choosing between these algorithms depends on specific needs regarding computational efficiency, key management, and security level.

5 Cryptographic Foundations of ECDSA

5.1 The Discrete Logarithm Problem

The rising prevalence of elliptic curve cryptography (ECC) is predominantly caused by the strength-per-key-bit, which is substantially greater in algorithms that use this mathematical concept. In return, the increased strength-per-key-bit and therefore higher security of ECC are based on the hardness of the elliptic curve discrete logarithm problem (ECDLP), which is assumed to be computationally intractable. The more general discrete logarithm problem (DLP), from which the ECDLP is derived, is also subject to other well-known cryptographical algorithms such as the Diffie-Hellman key exchange. It is defined as follows: Let G be a cyclic group of order n with the generator g . The discrete logarithm of $h \in G$ to the base g , denoted by $\log_g(h)$, is the unique integer k , $0 \leq k \leq n - 1$, such that $[k]g = h$ [4]. In broader terms, this essentially states that given g and h , find k such that $g^k = h$.

5.2 Elliptic curve group

In case of ECC, the cyclic group from the ECDLP can be thought of as a set of Points on an elliptic curve E where E represents the Weierstraß equation

$$y^2 = x^3 + ax + b, \quad a, b \in \mathbb{F}_p, \quad 4a^3 + 27b^2 \neq 0$$

The curve E is defined over a finite field $\mathbb{F}_p$ with p being prime. This means that only the points P where $x_P, y_P \in \mathbb{F}_p$ are of interest.

The set of points in E over $\mathbb{F}_p$ denoted by $E(\mathbb{F}_p)$ can then be written as

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 : y^2 = x^3 + ax + b\} \cup \{O\}$$

with O being the point at infinity and representing the group's identity element. It becomes clear why that is once the group operation of $E(\mathbb{F}_p)$ is discussed in section 5.2.2.

The cyclic group of order n is then built by a generator point $G \in E(\mathbb{F}_p)$ by repeatedly applying the group operation, often denoted as addition:

$$\langle G \rangle = \{[1]G, [2]G, [3]G, \dots, [n-1]G, [n]G\}$$

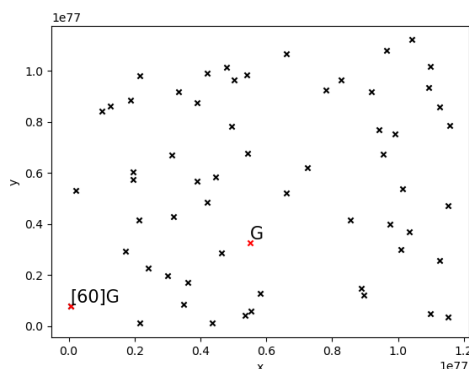
$[k]G$ for example translates to $G_1 + G_2 + \dots + G_{k-1} + G_k$, where '+' displays the group operation. Since E is defined over the finite field $\mathbb{F}_p$ the group law is performed within this field by reducing the points modulo p . This bounds the points on the curve to integer coordinates within a $p \times p$ matrix [4].

5.2.1 Example curve *secp256k1*

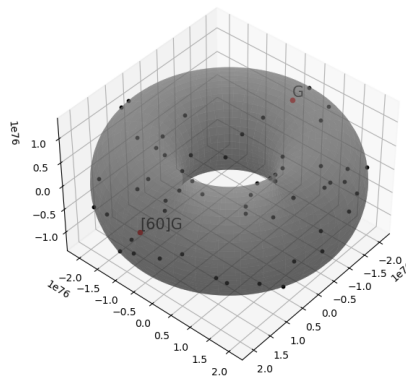
Suppose the elliptic curve *secp256k1* used in Bitcoin is given by the following domain parameters in hexadecimal, as proposed in [18]:

[illegible]

The corresponding curve can be seen in Figure 2. In the given setting, calculating $[k]G$ where $k = 60$, for example, generates a trace of seemingly randomly distributed points, as depicted in Figure 1a. Since both the x and y -coordinates of the points are reduced modulo p , they can also be thought of as points on a torus[6]. The torus arises by firstly reducing $x \bmod p$, which leads to a cylinder and then reducing $y \bmod p$ which finally results in the torus. This can be seen in Figure 1b.



(a) Points displayed in $p \times p$ matrix



(b) Points mapped onto torus

Figure 1: [1]G to [60]G on secp256k1

At its heart, the core principle of elliptic curve cryptography and therefore the ECDLP lies in the fact that given the results of $[1]G$ and $[k]G$, finding k is assumed to be infeasible. In a real-world setting such as the implementation of Bitcoin, k would be a potentially large 256-bit key, making an exhaustive search for its value completely inefficient.

5.2.2 Elliptic curve group operation

Let $P, Q \in E(\mathbb{F}_p)$ be points on the elliptic curve. The group operation, often denoted as addition $P + Q$, has a simple geometric interpretation, as can be seen in Figure 2. If one were to draw a line l through P and Q , then l would intersect the curve E at a third point denoted as $-R$. One then proceeds to reflect the point $-R$ around the x -axis. Since E is symmetric around the x -axis, the new reflected point R will also be on the curve. Hence, the result of the operation $P + Q$ is R . For the addition of two

Points, four different cases have to be distinguished. The following approach is based on the description found in [4]:

1. One of the points is the identity element O : $P + O = P$. The line l through P and O is vertical, since O is the point at infinity. Therefore, the point of intersection of l and E is the reflection of P , which means reflecting it one more time results in P itself.
2. The two points are not at infinity, nor are they identical, but they have the same x -coordinates. Additionally, one defines $O = -O$: $P + (-P) = O$. If the two Points have the same x -coordinates but are not identical, that means they have the coordinates $P = (x_P, y_P)$ and $-P = (x_P, -y_P)$. The point of intersection and therefore its reflection as well will be at infinity.
3. The two points are not at infinity, nor do they have the same x -coordinates: $P + Q = -R$. In this case the line through P and Q is not vertical which means that it intersects the curve at a third point $R \neq O$.
4. The two points are not at infinity, and they are identical: $P + P = -R$. The line l is the tangent to the curve E in point P and intersects E in another point R .

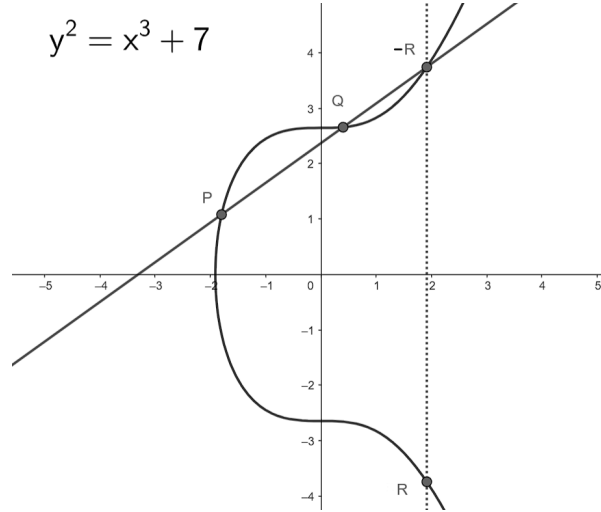


Figure 2: Group operation on elliptic curve *secp256k1* used in Bitcoin

5.2.3 Calculating R

In cases 1 and 2 of section 5.2.2, the resulting points after the group operation are immediately obvious. Only in cases 3 and 4 do the resulting points have to be actually calculated. To perform the group operation on points of these cases, the formulas for the x_r and y_r values of the resulting point R have to be derived.

Given two points $P \in E(\mathbb{F}_p)$ and $Q \in E(\mathbb{F}_p)$ that comply with case 3, the values for x_r and y_r of the resulting point $R \in E(\mathbb{F}_p)$ can be calculated as follows:

$$\lambda = \frac{y_p - y_q}{x_p - x_q}, \quad y - y_q = \lambda(x - x_q) \quad (1)$$

$$y = \lambda x - \lambda x_q + y_q, \quad \beta = y_q - \lambda x_q, \quad y = \lambda x + \beta$$

$$(\lambda x + \beta)^2 = x^3 + ax + b$$

$$0 = x^3 - \lambda^2 x^2 - 2\lambda x\beta - \beta^2 + ax + b$$

using Vietas formula for cubic polynomials we get:

$$x_r = \lambda^2 - x_p - x_q$$

and for y after plugging in x in 1:

$$y_r = \lambda(x_p - x_r) - y_p$$

In case 4, where P and Q are identical, the procedure for calculating λ changes. The slope is now obtained by taking the derivative of E :

$$\frac{dx}{dy} = \frac{d}{dy}(x^3 + ax + b)^{\frac{1}{2}}$$

$$\frac{dx}{dy} = \frac{3x^2 + a}{2\sqrt{x^3 + ax + b}}$$

and since $y^2 = x^3 + ax + b$ we can just substitute y back in:

$$\lambda = \frac{3x^2 + a}{2y}$$

5.3 Key Pair Generation

Since the ECDSA operates in the realm of asynchronous cryptography, it relies on key pairs consisting of a public key P and a private key d . In the case of ECDSA, the objective is to prove the authenticity of the sender. Therefore, the sender possesses a private key, which is used to sign the message being sent. Additionally, there exists a public key, that recipients of the message can use to verify that the message was signed with the corresponding private key, ensuring it came from the expected sender. In ECDSA, the private key is an integer d randomly chosen from the interval $\{1, 2, \dots, n - 1\}$ where n is the group order of the cyclic group G . The public key P is then generated by applying the group operation on the generator G exactly d times. This procedure is structured based on the methodology described in [8]:

1. choose private key $d = \text{random}(\{1, 2, \dots, n - 1\})$
2. calculate public key $P = [d]G$

3. output key pair (d, P)

One might argue that the calculation of the public key P in step 2 could be computationally expensive, as the group order n is usually a large prime number, potentially resulting in a very large value for d . Thankfully, there exists an efficient algorithm for computing scalar multiplication in groups with an operation that is both associative and commutative, called double-and-add. Instead of iterating over all numbers from 0 to k - in other words building k by iteratively adding 1 - one can build up k by either adding 1 to the current number or doubling the current number. For example, if $k = 11$, instead of calculating $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 11$, one could just calculate $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 10 \rightarrow 11$. But how does one determine when to add 1 and when to double? The Solution is to start from k and divide by 2 when the current number is even, and subtract 1 when it is odd[16]. This demonstrates that the fact that $E(\mathbb{F}_p)$ carries group structure, such that its group operation is both associative and commutative, plays a crucial role in making elliptic curve cryptography applicable in the first place.

5.4 Signature Generation

The Signature Generation process is typically performed by the sender of some data, denoted as a message M , to provide a way of proving ownership of that particular resource, as well as ensuring its integrity and authenticity. This is achieved by signing the message M utilizing the sender's private key d , created in the key pair generation step. The owner signs the message in such way that the signature can be verified by the recipients using the public key generated in the key pair generation step as well. The algorithm including the pseudocode presented herein follows the narrative detailed in [8]:

1. choose $k = RNG(\{1, 2, \dots, n - 1\})$

The first step is to choose a random integer k between 1 and $n-1$ which specifies the number of times the group operation is applied to the generator G in the second step. It is of utmost importance to choose a cryptographically secure random number for k , or else the private key can be deduced.

2. calculate $Q = [k]G$

The second step consists of applying the group operation, as discussed in sections 5.2.2 and 5.2.3 to the generator G k times. As mentioned in section 5.3, in order to mitigate the computational cost of this operation, the double-and-add algorithm is used for scalar multiplication.

3. calculate $r = x_Q \mod n$

The x -coordinate of Q is taken as part of the final signature. To conform the requirement that the values fall within the expected range, is it reduced modulo n as well.

4. calculate $k_{inv} = k^{-1} \mod n$

This step involves calculates the multiplicative inverse of $k \mod n$, which will be used in the final calculation in step 5.

5. calculate $s = k_{inv} * (r * d_A + H_{L_n}(M)) \mod n$

Here the second and therefore final component s of the resulting signature is calculated. This can be seen as the actual signing step. d_A is the private key of the signer that was obtained in the key pair generation step. $H_{L_n}(M)$ first hashes the message M to be signed, using a cryptographic hash function such as $SHA - 256$. It then takes the L_n leftmost bits of that hash. Here, L_n stands for the number of bits of the group order n . Finally, the value is being converted to an integer. The hashing adds an additional layer of security as it ensures the integrity of the message M . The entire term is then reduced modulo n again to comply with the policy of the group order n .

6. Output (r, s)

The final output of the signature algorithm is a pair of values r and s .

5.5 Signature Verification

The signature verification process is performed by the receivers of our message M and ensures the integrity and authenticity of this message or transaction. During verification, a recipient or verifier utilizes the signer's public key from the key pair generation step as well as the received signature from the signature generation step to authenticate the origin and integrity of the message. If the signature is valid, it provides cryptographic assurance that the message was indeed signed by the purported signer using their private key and that the message has not been altered during transmission. The following algorithmic approach, including the pseudocode, is outlined in [8]:

1. verify that $r, s \in \{1, 2, \dots, n - 1\}$

This requirement follows directly from the fact that r and s were calculated modulo n in the signature process. If s and r are not in the interval, the signature is invalid.

2. calculate $z = H_{L_n}(M)$

$H_{L_n}(M)$ has to calculate the hash of M the same way as in step 5 of the signature generation. This step is necessary to verify that the hash value of the received message matches the hash value received from the sender, indicating that the message has not been altered during transmission.

3. calculate $s_{inv} = s^{-1} \mod n$

This step calculates the multiplicative inverse of s modulo n . For the calculation, the extended Euclidean algorithm can be used.

4. calculate $u_1 = s_{inv} * z \mod n$ and $u_2 = s_{inv} * r \mod n$

This is an intermediate step that utilizes the values from steps 2 and 3 to calculate u_1 and u_2 .

5. $Q = [u_1]G + [u_2]P_A$

Finally, calculate Q by applying double-and-add scalar multiplication between u_1

and the generator G , as well as u_2 and the signer's public key P_A . The obtained value for Q is congruent modulo n with Q from step 2 of the signature generation process if the signature is valid. The correctness of this calculation can be proven by simply plugging back in the values from steps 2, 3 and 4, as shown in [8] as well:

- (a) $kG = u_1G + u_2P_A$
- (b) $kG = s_{inv}zG + s_{inv}rP_A$
- (c) $kG = s_{inv}zG + s_{inv}rd_AG$
- (d) $kG = s_{inv}G(z + rd_A)$
- (e) $s = k_{inv}(H_{L_n}(M) + rd_A)$

where s of step (e) is equivalent to s of step 5 of the signature generation.

6. $v = x_Q \bmod n$

As in step 3 of the signature generation, only the x -coordinate of point Q will be used for the final verification.

7. Output *true* if $v = r$, and *false* otherwise

The signature verification algorithm returns *true* only if v from step 6 is equal to the signature value r which is the x -coordinate of point Q from the signature generation process.

5.6 ECDLP vs DLP - why it is more secure

Trying to explore the security of elliptic curve cryptography can be challenging because there exists no definite proof of its security. It seems as if over the years, the cryptographic community gained trust in the robustness of ECC because of it being exposed to all the world's top-tier hackers and cryptographers for a long time. That, of course, implies complying with the specifications for standardized curves and cryptographically strong domain parameters[4]. One reasonable approach to understanding why elliptic curve cryptography is actually secure is to recognize that the currently best-known algorithms for solving the DLP are either impractical or even inapplicable to the ECDLP. As of now, the two fastest algorithms for solving the DLP are called Number Field Sieve and Index Calculus Method. They can achieve subexponential time complexities[11][14], while currently, no such algorithm exists for solving the ECDLP. A natural subsequent step would be to discuss the currently best-known algorithms to solve the ECDLP and why they have a higher time complexity than the algorithms known for the DLP. We will focus on general attacks that do not exploit any kind of special properties or additional knowledge within the given elliptic curve setting. We will only explain the most effective attacks against the ECDLP in more detail.

5.6.1 Baby Step Giant Step

The BSGS attack solves the ECDLP in $O(\sqrt{n})$ time -and space complexity, making it relatively memory exhaustive compared to the other algorithms. The algorithm described

herein is based on the information provided in [17]. Let $\langle G \rangle$ be a cyclic group with the generator G . Suppose G has order n , and let $P \in \langle G \rangle$. Additionally define $m = \lceil \sqrt{n} \rceil$. Suppose the ECDLP we want to solve consists of finding x such that $[x]G = P$. According to the Euclidean algorithm, we can then rewrite x (like any other integer as well) as a linear combination $x = m * i + j$ with $i, j \in [0, m - 1]$. The clue behind BSGS is now, thanks to the new representation of x , we can split up $[x]G$ into the two separate terms $[x]G = [mi]G + [j]G$ and therefore have $[mi]G + [j]G = P$. We now no longer have to check every $x \in [0, n]$ but now only check for $i, j \in [0, \lceil \sqrt{n} \rceil - 1]$. After rearranging this equation to $[j]G = P - [mi]G$ the main calculation starts. We first calculate the list of 'baby steps' $[j]G$ for $0 \leq j < m$ and store it (hence the space complexity of $O(\sqrt{n})$). We then proceed to calculate the 'giant steps' $P - [mi]G$ for $0 \leq i < m$ and each time check if the value is contained in the stored list. If a match is found, so is x since we then know i and j , while m is a constant anyway, and we therefore can compute $x \equiv m * i + j \pmod n$. Since we iterate over $0 \leq j, i < m$ respectively while $m = \sqrt{n}$, the time complexity is $O(2 * (\sqrt{n} - 1)) = O(\sqrt{n})$ [17].

5.6.2 Pollard's Rho-Algorithm

The currently best known general attack on the ECDLP is called the Pollard's Rho method. The standard version of this algorithm has the same asymptotic time complexity as the BSGS algorithm, but a much smaller space complexity. The following methodology is based on the insights provided in [17]. Let $\langle G \rangle$ again be a cyclic group. The ECDLP is the same as before: given $G, P \in \langle G \rangle$ find x such that $[x]G = P$. In the Pollard's rho algorithm we would not directly try to solve the equation $[x]G = P$ but instead try to find integer pairs (a, b) and (a', b') such that $[a]G + [b]P = [a']G + [b']P$. If such pairs are found, the value for x can be deduced:

$$[a]G + [b]P = [a']G + [b']P$$

$$[a]G - [a']G = [b']P - [b]P$$

$$[a - a']G = [b' - b]P$$

$$[a - a']G = [b' - b][x]G$$

In order to remove G here and then solve for x , we need to remember that since we operate within a cyclic group of order n , the endpoints on both sides of '=' will be the same but, the cofactors are not necessarily identical. They just need to be congruent modulo n to end up at the same point when multiplying with G :

$$(a - a') \equiv (b' - b)x \pmod n$$

$$x = (a - a')(b' - b)^{-1} \pmod n$$

To find such pairs the Pollard's Rho method proposes the following core principle: Define a function f that generates a random walk $x_1, x_2, x_3, \dots$ through $\langle G \rangle$ by repeatedly applying $x_{i+1} = f(x_i)$. For example, x_5 is generated by calculating $f(f(f(f(x_1))))$.

There is actually no fixed definition for this function f , some have just proven to be faster than others. The only requirement is that f generates pseudo-random points in $\langle G \rangle$ to enable the random walk. One such function is also defined in [17]. Probably the most important observation in the given setting is now that since we calculate the points in $\langle G \rangle$ modulo n , in other words, the group order of $\langle G \rangle$ is n , the random walk with f will eventually end up at a point it already visited. This means that there is a cycle in this random walk which looks the Greek letter ρ . The objective is now to find that circle since that would mean we obtained the two integer pairs (a, b) and (a', b') which result in the same point, hence we can deduce x as described above. This point is called collision. To find that collision, the Pollard's Rho algorithm makes use of another algorithm called Floyd's cycle-finding algorithm. The trick of this algorithm is to have a slow tortoise that always calculates $f(x)$, for example, and a fast hare that always calculates $f(f(x))$, for example. If we let them both run through the cyclic group $\langle G \rangle$ at their different speeds, they will eventually collide in the circle, which means we found the collision and can compute x . This shows that the space complexity of the Pollard's Rho algorithm is just $O(1)$ compared to $O(\sqrt{n})$ of the BSGS algorithm. As already mentioned, the time complexity of this standard version of the algorithm is $O(\sqrt{n})$ as well. Though this time complexity can be reduced to $O(\sqrt{\frac{n}{q}})$ with q being the largest prime factor of the order of the group. This is achieved by introducing multithreading such that computing the random walk happens in parallel [17].

6 Security Risks

The ECDSA is a clever way of keeping our digital world secure. But, if we want to use ECDSA effectively, we need to pay attention to some of the challenges it brings, especially when it comes to how difficult it can be to implement because it's quite complex. ECDSA's strength comes from the complicated math it is based on, and it uses smaller but tough-to-crack keys. This helps to save on data space. However, this complexity can also make ECDSA tough to put into practice. If not implemented correctly, it can easily lead to security issues.

ECDSA has a lot of possible issues. For example, the parameters for elliptic curves have a lot of restrictions to be secure. In the following sections, we're going to restrict ourselves to two major challenges: The potential pitfalls with bad random number generators and the possibility of time-based attacks. We'll also provide possible solutions to mitigate these issues. Furthermore, we will present real-world examples of where such attacks have been successfully executed, providing a practical understanding of these concepts.

It should be noted that we used ChatGPT to generate most of the text in this section. Still, the outline was done beforehand and the information provided by ChatGPT has been checked to be correct. A lot of text ChatGPT generated has been cut, and some handwritten parts have been added for further explanation. We used footnotes to document where we had to correct ChatGPT, or we didn't use it at all. Where applicable, we added citations of fitting scientific papers which were researched beforehand.

6.1 Poor Random Number Generator

The nonce (number used once) is a crucial component of ECDSA and contributes a significant layer of security. It must be unique and random for every signature to ensure the private key isn't revealed. However, the security of these nonces and, therefore, the overall encryption system heavily depends on the strength of the random number generator used. There are four key issues we need to consider while discussing the quality of random number generators for nonces: repetition, predictability, size and bias.¹³

6.1.1 Repetition

Reusing a nonce in multiple signatures is a significant security flaw. Even though the messages signed might be different, the fact that the same nonce was used provides an attacker with a critical amount of information. With two signatures sharing the same nonce but different messages, the private key can be calculated. [5]

To understand this, consider the ECDSA signature equation¹⁴ from section 5.4:

$$s = \frac{h(m) + rd}{k} \bmod q \quad (2)$$

In this equation, k represents the nonce, $h(m)$ is the hash of the message, d is the private key, r is the x-coordinate of the EC point $r = kG$, and q is the order of the elliptic curve's generator point G . If the same nonce k is used for two messages, $m1$ and $m2$, resulting in two signatures $(r, s1)$ and $(r, s2)$, notice that r , the x-coordinate, is the same for both since the same k was used.

This presents a problem as it allows any observer to calculate the nonce k . This can be done by solving equation 2 for $r d$. For both signatures this value is equal. This results in the following equation for k :

$$k = \frac{h(m1) - h(m2)}{s1 - s2} \bmod q \quad (3)$$

The hashed message $h(m)$ and the signature s is known and the nonce k can be calculated. Once the nonce is known, the observer could afterwards calculate the private key d using the private key equation derived from equation 2:

$$d = \frac{sk - h(m)}{r} \bmod q \quad (4)$$

All the values in this equation are now known and the private key d is revealed. This means that the reusing of nonces can lead to the exposure of the private key. It's crucial to ensure that nonces are unique and not reused when generating ECDSA signatures to

¹³Starting out, ChatGPT was overly concerned with predictability of generated keys. This is still important, but it is not a unique problem of random number generators to ECDSA. We had to clarify the issues with bad nonces.

¹⁴Surprisingly ChatGPT generated these mathematical equations correctly. Only the formatting had to be adjusted to get nice looking equations.

maintain a secure system. This includes the use of a proper random number generator, that cannot produce the same number twice.

6.1.2 Predictability

If an attacker can predict the next number generated from a specific algorithm, it instantly voids the security of the cryptographic operations relying on that algorithm. ECDSA, in particular, relies on the randomness of the nonces generated. Predictable nonces can enable an attacker to reverse-engineer and reveal the private key with equation 4 in section 6.1.1. This vulnerability drastically undermines the effectivity of the ECDSA security protocol.

In parallel, the same principle applies to the generation of keys. Keys should ideally be unique and unpredictable. However, if the pseudo-random number generator used for the key creation is compromised, it results in a production of predictable keys, rendering the entire security protocol ineffective. An attacker could potentially predict future keys and gain unauthorized access to sensitive data. However, this concept is not unique to ECDSA and is not further explored in this paper. A solution to this issue would be to use a random number generator that makes it impossible to predict its output.

6.1.3 Size

When nonce values are of small sizes such as 64-bit or 128-bit, they are susceptible to cryptanalytic attacks. One possible attack is using Pollard’s rho algorithm as outlined in section 5.6.2 to solve for these small nonces. The algorithm can determine the discrete logarithm—that is, find the nonce—in a number of steps proportional to the square root of the size of the group. 64-bit or 128-bit nonces are small enough for a targeted attack to be feasible [2]. A prevention to this attack is to simply to use nonces that are bigger, for example 256-bit.

6.1.4 Bias

Sometimes there might be patterns or ‘biases’ in how the nonce is created. If these biases exist, they could potentially hint at what the nonce number is. If someone else can guess this nonce, they might infringe on our coded messages’ security. Therefore, it is vital to understand these biases, their impacts, and how to avoid them in simple and practical terms. We will delve into this topic in the following section.

In the ideal scenario, each nonce generated in ECDSA should be randomly unique. This means that it does not correlate with any other nonce and has no recognizable patterns. Ensuring complete randomness means the security of the system remains intact. However, if a part of the nonce remains the same across different computations — say, the first few or the last few digits don’t change — this introduces a bias. This is problematic because it gives an attacker something to work with. By analyzing multiple signatures and exploiting these consistencies, they could derive the private key using these biases in the nonce generation. This can be achieved with the Hidden Number Problem (HNP).

The HNP involves guessing a hidden number when you only have partial information or hints. Imagine you have a long number, but you only know some of its digits and the position of these digits in the number. The hidden part of the number would be the digits you do not know. The challenge of the HNP is to figure out these unknown digits. [13]

Relating it back to our scenario, an attacker who can determine a part of the nonce due to a bias can use this knowledge as a hint. If they can obtain multiple signatures that contain these weak nonces, they can create a system of equations satisfying the HNP. By solving these equations, they aim to discover the unknown digits. In literature there are two techniques to solve the HNP: Lattice-based techniques and Fourier analysis [2]. The following paragraphs will explain how the former technique is used for extracting a key from a bias in the nonces. Fourier analysis is more complicated, but it is more useful for more samples and smaller biases.

In simple terms, a lattice is a grid formed by points in space. This grid is defined by a set of basis vectors. The main goal of lattice reduction is to find a new and shorter basis for the same lattice. When applied to the Hidden Number Problem, the aim is to use these simplified or reduced lattices to make the problem more manageable.

So, let's say we have a series of signatures with a bias in their nonce values making a part of each nonce predictable. These partial nonces, along with the known signatures and public key, can be used to create a lattice in multidimensional space. In this lattice the solution is represented as a short vector. This vector is known to be close to a lattice point but not exactly on it. To find this short vector, a Lattice Basis Reduction algorithm is used, such as Lenstra-Lenstra-Lovász (LLL) algorithm. [2, 13]

This results in a candidate value for the nonce and a possible private key can be calculated with the equation 4. With this key the public key can be calculated, and it can be compared with the actual public key. If the keys match, the private key is revealed [2]. More reading can be done on how the lattice is constructed in [13].

It should be noted, that a non-uniform distribution of the nonce generation can also introduce a bias that can be exploited with this technique. Also, it's worth mentioning that biases in nonces or even similar nonces can result in multiple security risks which include two different actors being able to find out the private key of the other one with their own private key. How these risks work is outlined in [5]. Therefore, it is crucial for the random number generator to be uniform.

6.1.5 Examples

The pitfall of repeated nonces has occurred a few times in the past.¹⁵ At one point in the past, Sony used a fixed nonce to sign their software for the PlayStation 3 [9]. Also, the Java secure random generator had issues with the random number overlapping sometimes [15]. As outlined in section 6.1.1, this allows for the private key to be calculated. In Sony's example this means that others could use the private key to sign their own software.

¹⁵We wanted to use our own examples we found while researching and didn't use ChatGPT for this section.

Java's number generator was implemented in some Android applications. This resulted in the leakage of various keys of bitcoin wallets [5].

The exploitation of biases in nonces, or even small nonces, is explored in [2]. Biases in nonces, and even small nonces, occurred in faulty ECDSA implementations used in cryptocurrency transactions. These issues can be exploited and some addresses of cryptocurrency wallets were already compromised. This can be done as described in section 6.1.4. There are a few categories the faulty nonce generation fall into. First, there are very small nonces with 32 bits or fewer. Most of these nonces appear to be handcrafted. The next category are small nonces ranging from 64-bit to 160-bit. Also, 256-bit nonces that share 128-bit suffixes have been exploited to find the private key used to sign the transaction with these faulty nonces. Of course this could be expanded or modified to exploit even smaller biases. [2]

6.1.6 Solution

As seen in this section about nonce generation it's important the nonce doesn't leave any security risks. The solution to this issue is to use a cryptographically secure pseudorandom number generator. This ensures that the random number can't be predicted or backtracked, the values don't repeat and there is no bias in the numbers. Possible random number generators usable for proper nonce generation are EdDSA3 or RFC6979 [9]. This ensures there isn't any possible prediction, repetition or bias in the generated nonces. Using a random number generator that doesn't ensure these issues aren't a problem will leave a huge security risk to the application using this faulty implementation of ECDSA.

6.2 Timing Attack

One possible type of attack that can happen is a timing attack. This attack focuses on how long it takes for the algorithm to work. By studying the time it takes to process information, the attacker might be able to find confidential information that compromises the key. This section is based on [3] and will look at how this type of attack works on ECDSA and how it can be prevented.¹⁶

6.2.1 Key Extraction

A timing attack could be possible because of a faulty implementation of the Montgomery Ladder for the ECDSA. It is a method used to carry out scalar multiplication on the elliptic curve. A simple explanation is found in section 5.3. The algorithm iterates through the bits of the scalar and uses an efficient solution to double the number or add 1 to the number. This solution also takes the same amount of time regardless of if the bit is a 1 or a 0. This algorithm starts at the most significant bit that is a 1. The initial 0-bits have to be skipped because those don't have an effect on the point of the curve. An operation cannot be executed on the leading 0-bits.

¹⁶ChatGPT is stupid and wasn't used in this section because it's really not that hard.

This poses a problem because this means the algorithm doesn't run in constant time anymore. An attacker could take measurements of how long the signature generation takes. In this step a random nonce is used for scalar multiplication. If the leading bits of the nonce are 0 then the algorithm takes shorter to execute than expected because the leading 0-bits are ignored. With the multiple measurements the exact number of 0-bits can be figured out. This means that a certain prefix is known for a nonce associated with a signature. This Prefix is a bias and can be taken advantage of with a lattice attack as described in section 6.1.4.

Specifics on how the time is measured and how the lattice attack is performed can be found in the above-mentioned paper that this section is based on. It should be noted that the lattice attack does allow for some error in the data and that with enough measurements the private key can be successfully revealed. The paper found this vulnerability in OpenSSL and provides a fix for the implementation. It was included in OpenSSL 1.0.0e.¹⁷

6.2.2 Solution

As outlined in the previous section the leading 0-bits can't be ignored because otherwise the algorithm wouldn't run in constant time which allows a side-channel attack. A solution to this issue is to guarantee that the scalar is a constant size. The oversimplified explanation is to add a 1 in front of the scalar to ensure the 0-bits are not ignored. In practice this process is slightly more complicated because this would not result in the correct point on the curve. Instead, the order of the curve is added to the scalar. If the resulting scalar still has the size of the order, the order is added another time. This results in a scalar that is always the same size. This is also the correct point on the curve because adding the order of the curve to a point on the curve results in the point itself.

7 Conclusion

The ECDSA is a very powerful tool in cryptography. The digital signature allows for a sender of a message to be verified, for example for cryptocurrency transactions or for web applications. With ECC this can be performed even with low computational power and smaller key lengths are required for the same level of security achieved with RSA. Still, its math is relatively more complicated and a lot of care should be taken when implementing elliptic curves. If not implemented correctly, this results in security flaws that compromises the authenticity of the sender. However, if proper care is taken and time is spent on understanding ECC, ECDSA is a great choice for digital signatures.

¹⁷OpenSSL Changelog: <https://www.openssl.org/news/changelog.txt>

References

- [1] Mishall Al-Zubaidie, Zhongwei Zhang, and Ji Zhang. Efficient and secure ecdsa algorithm and its applications: A survey. *arXiv preprint arXiv:1902.10313*, 2019. <https://arxiv.org/pdf/1902.10313>.
- [2] Joachim Breitner and Nadia Heninger. Biased nonce sense: Lattice attacks against weak ecdsa signatures in cryptocurrencies. Cryptology ePrint Archive, Paper 2019/023, 2019. <https://eprint.iacr.org/2019/023>.
- [3] Billy Bob Brumley and Nicola Tuveri. Remote timing attacks are still practical. Cryptology ePrint Archive, Paper 2011/232, 2011. <https://eprint.iacr.org/2011/232>.
- [4] Bundesamt für Sicherheit in der Informationstechnik (BSI). Technical Guideline TR-03111: Elliptic Curve Cryptography, Version 2.0. Technical Report, 2012.
- [5] Nicolas T. Courtois, Pinar Emirdag, and Filippo Valsorda. Private key recovery combination attacks: On extreme fragility of popular bitcoin key management, wallet and cold storage solutions in presence of poor rng events. Cryptology ePrint Archive, Paper 2014/848, 2014. <https://eprint.iacr.org/2014/848>.
- [6] Gaël Foppolo. Elliptic curve cryptography (ecc), 2018. Accessed: May 7, 2024.
- [7] Jiří Gavenda. Threshold ecdsa performance analysis. Master’s thesis, Masaryk University, Brno, Spring 2021. <https://is.muni.cz/th/fuwu0/thesis.pdf>.
- [8] Yasin Genç and Erkan Afacan. Design and implementation of an efficient elliptic curve digital signature algorithm (ecdsa). In *2021 IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS)*, pages 1–6, 2021.
- [9] Jan Jancar, Vladimir Sedlacek, Petr Svenda, and Marek Sys. Minerva: The curse of ecdsa nonces. Cryptology ePrint Archive, Paper 2020/728, 2020. <https://eprint.iacr.org/2020/728>.
- [10] Don Johnson, Alfred Menezes, and Scott Vanstone. The elliptic curve digital signature algorithm (ecdsa). *Certicom Research, Canada & Dept. of Combinatorics & Optimization, University of Waterloo, Canada*, 2001. Available at: <https://www.cs.miami.edu/home/burt/learning/Csc609.142/ecdsa-cert.pdf>.
- [11] A. K. Lenstra, H. W. Lenstra, M. S. Manasse, and J. M. Pollard. The number field sieve. In *Proceedings of the Twenty-Second Annual ACM Symposium on Theory of Computing*, STOC ’90, page 564–572, New York, NY, USA, 1990. Association for Computing Machinery.
- [12] Sharon Levy. Performance and security of ecdsa. *Comput. Sci*, 2015.

- [13] Gabrielle De Micheli and Nadia Heninger. Recovering cryptographic keys from partial information, by example. Cryptology ePrint Archive, Paper 2020/1506, 2020. <https://eprint.iacr.org/2020/1506>.
- [14] R. Padmavathy and Chakravarthy Bhagvati. Discrete logarithm problem using index calculus method. *Mathematical and Computer Modelling*, 55(1):161–169, 2012. Advanced Theory and Practice for Cryptography and Future Security.
- [15] Damian Poddebniak, Juraj Somorovsky, Sebastian Schinzel, Manfred Lochter, and Paul Rösler. Attacking deterministic signature schemes using fault attacks. Cryptology ePrint Archive, Paper 2017/1014, 2017. <https://eprint.iacr.org/2017/1014>.
- [16] Jacek Samotyja and Kerstin Lemke-Rust. Practical results of ecc side channel countermeasures on an arm cortex m3 processor. In *Proceedings of the 2016 ACM Workshop on Theory of Implementation Security*, TIS '16, page 27–35, New York, NY, USA, 2016. Association for Computing Machinery.
- [17] Joseph Spring. A case of standard attacks against the dlp and ecdlp. 01 2003.
- [18] Mohammed Mujeer Ulla, Preethi, Md. Sameeruddin Khan, and Deepak S. Sakkari. Implementation of elliptic curve cryptosystem with bitcoin curves on secp256k1, nist256p, nist521p, and lll. *Journal of ICT Standardization*, 11(4):329–353, 2023.
- [19] Chaitanya Varma. A study of the ecc, rsa and the diffie-hellman algorithms in network security. In *2018 International Conference on Current Trends towards Converging Technologies (ICCTCT)*, pages 1–4, 2018.
- [20] Zuguang Xuan, Zhenjun Du, and Rong Chen. Comparison research on digital signature algorithms in mobile web services. In *2009 International Conference on Management and Service Science*, pages 1–4, 2009.

List of Figures

1	[1]G to [60]G on secp256k1	9
2	Group operation on elliptic curve <i>secp256k1</i> used in Bitcoin	10